Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Databricks + Data Engineering + Data Engineer + Big Data + Pyspark +

Databricks Level 3 (Transformation Layer) — Turning Raw Data into Reliable Business Truth with SQL, PySpark, Incremental Processing, and Data Modelling



Anurag Sharma 17 min read · Oct 5, 2026

63 1 1



Summary

Level 3 (Transformation Layer) is where raw data becomes useful, trusted, and business-ready. If Level 2 (Lakehouse Foundations) gave you Delta tables, Medallion architecture, and ingestion pipelines, Level 3 is where you decide how data should be cleaned, standardized, enriched, modeled, deduplicated, and delivered to downstream consumers. This blog goes deep into production SQL patterns, PySpark transformation design, incremental processing with `MERGE`, handling late-arriving data, slowly changing dimensions, dimensional modeling, and the transformation mistakes that often surface in senior interviews and production failures. This is the layer where data engineers stop being file movers and start becoming builders of business truth.

Who Should Read This Blog

1. Data engineers who can ingest data but struggle to structure transformation logic

2. Engineers moving from ETL tools into Databricks and modern lakehouse workflows
3. Anyone preparing for Databricks, Spark, or senior data engineering interviews
4. Developers writing SQL or PySpark transformations that are becoming hard to maintain
5. Tech leads trying to standardize Bronze → Silver → Gold transformation ownership
6. Engineers who want to move from “jobs that run” to “data products people trust”

Why Level 3 (Transformation Layer) Matters So Much

A lot of engineers think data engineering is about moving data from one place to another.

It isn’t.

Moving data is logistics.

Transformation is where engineering judgment begins.

This is the layer where you answer questions like:

- Which records are valid?
- Which duplicates should survive?
- What makes one customer record more trusted than another?

- When an upstream schema changes, do we absorb it, reject it, or quarantine it?
- How do we preserve history?
- How do we make the same logic re-runnable without producing different outputs?
- How do we model data so analysts, dashboards, finance teams, and ML systems all use the same truth?

If Level 2 (Lakehouse Foundations) protects the platform from becoming a swamp, **Level 3 (Transformation Layer) protects the business from becoming confused.**

Bad ingestion creates technical problems.

Bad transformation creates **trust problems.**

And trust problems are harder to fix.

The Real Job of a Transformation Layer

The transformation layer is not just “where we clean data.”

That definition is too small.

A mature transformation layer should do all of the following:

1. **Validate** incoming records
2. **Standardize** names, types, formats, units, and codes

3. **Deduplicate** conflicting or repeated data
4. **Conform** data across systems into shared definitions
5. **Enrich** records with reference/master data
6. **Model** datasets for analytics, reporting, and ML
7. **Preserve business history** where needed
8. **Support incremental processing**, not full reloads by default
9. **Handle late-arriving and out-of-order data**
10. **Produce trusted datasets with stable semantics**

This is why Level 3 (Transformation Layer) is where many teams either become disciplined — or collapse into notebook sprawl and logic duplication.

Step 1: Bronze, Silver, Gold — What Transformations Belong Where

One of the biggest causes of chaos in Databricks projects is not technical complexity.

It is **unclear layer responsibility.**

Let's define that clearly.



Bronze

Bronze should contain **raw data plus provenance**, not business logic.

Allowed in Bronze:

- Light parsing required for ingestion
- Metadata enrichment (`_source_file` , `_ingest_timestamp` , `_batch_id`)
- Minimal normalization needed to land data safely

Not allowed in Bronze:

- Business rule enforcement
- Deduplication based on business meaning
- PII decisions
- Revenue calculations
- KPI derivation

- Reference joins
- Gold-style transformations

Silver

Silver is the t**rusted, conformed, entity-level layer.**

This is where transformations should usually happen:

- Type casting
- Null handling
- Deduplication
- Standardization
- Joins with trusted reference data
- Validation rules
- Entity resolution
- PII treatment
- Slowly changing dimensions
- Record survivorship logic

Gold

Gold is the **consumption layer.**

Gold transformations should optimize for:

- Business reporting
- Dashboard consumption

- KPI aggregation
- Feature tables
- Department-specific marts
- Denormalized read models

Simple rule

If you are asking:

- “Is this record trustworthy?” → Silver
- “How should the business consume it?” → Gold
- “How did it arrive?” → Bronze

That one distinction alone prevents a lot of messy architectures.

Step 2: SQL Is Still the Core Language of Transformation

People sometimes over-romanticize PySpark.

But in real production environments, **SQL remains the clearest and most maintainable transformation language for a huge percentage of workloads.**

Why SQL remains dominant in Level 3 (Transformation Layer):

- More readable for mixed teams
- Easier for analysts and engineers to review together
- Strong optimizer support

- Easier lineage tracking
- Great fit for joins, aggregations, filtering, and window functions
- Easier code reviews than deeply nested DataFrame chains

A strong data engineer should be able to write production-grade SQL, not just ad hoc SQL.

Example: a weak SQL transformation

```
SELECT *
FROM bronze.orders
WHERE status IS NOT NULL
```

This is not a transformation design. It is a filter.

Example: a stronger SQL transformation pattern

```
WITH source AS (
    SELECT *
    FROM bronze.orders
    WHERE _ingest_date = current_date()
),

typed AS (
    SELECT
        CAST(order_id AS STRING)              AS order_id,
        CAST(customer_id AS STRING)           AS customer_id,
        CAST(order_timestamp AS TIMESTAMP)    AS order_timestamp,
        CAST(order_amount AS DECIMAL(12,2))   AS order_amount,
        UPPER(TRIM(status))                   AS status,
        CAST(order_channel AS STRING)         AS order_channel,
        _source_file,
        _ingest_timestamp
    FROM source
),
```

```
validated AS (
    SELECT *
    FROM typed
    WHERE order_id IS NOT NULL
      AND customer_id IS NOT NULL
      AND order_timestamp IS NOT NULL
      AND order_amount >= 0
      AND status IN ('PLACED', 'SHIPPED', 'DELIVERED', 'CANCELLED')
),

deduplicated AS (
    SELECT *
    FROM (
        SELECT
            *,
            ROW_NUMBER() OVER (
                PARTITION BY order_id
                ORDER BY _ingest_timestamp DESC
            ) AS rn
        FROM validated
    ) x
    WHERE rn = 1
)

SELECT
    order_id,
    customer_id,
    order_timestamp,
    order_amount,
    status,
    order_channel,
    _source_file,
    _ingest_timestamp
FROM deduplicated
```

This pattern is much stronger because it separates:

- source selection
- typing
- validation
- deduplication
- final projection

That structure makes debugging and reviewing much easier.

Step 3: The 7 Core Transformation Patterns Every Engineer Should Know

At Level 3 (Transformation Layer), most transformation logic falls into a handful of recurring patterns.

1. Standardization

Bring inconsistent representations into one format.

Examples:

- `usa`, `US`, `United States` → `USA`
- `paid`, `Paid`, `PAID` → `PAID`
- mixed date formats → standard timestamp/date types

```
SELECT
    customer_id,
    UPPER(TRIM(country_code)) AS country_code,
    TO_DATE(order_date, 'yyyy-MM-dd') AS order_date
FROM bronze.customers
```

2. Null handling

You should never casually replace nulls. Nulls mean something.

Use `COALESCE` only when the business meaning is valid.

```
SELECT
    order_id,
    COALESCE(discount_amount, 0.00) AS discount_amount
FROM silver.orders
```

Good use:

- numeric measures where missing truly means zero

Bad use:

- replacing missing `customer_email` with `'unknown'` and pretending it is valid

3. Deduplication

Not all duplicates are identical.

There are multiple duplicate patterns:

- exact row duplicates
- repeated event ingestion
- multiple records for same business key
- competing records from multiple systems

Common SQL pattern:

```
WITH ranked AS (
    SELECT
        *,
```

```
            ROW_NUMBER() OVER (
                PARTITION BY customer_id
                ORDER BY updated_at DESC, _ingest_timestamp DESC
            ) AS rn
        FROM silver.customer_updates
    )
    SELECT *
    FROM ranked
    WHERE rn = 1
```

This is not just deduplication. It is survivorship logic.

You are deciding which record wins.

4. Reference enrichment

Attach trusted lookup/master data.

```
    SELECT
        o.order_id,
        o.customer_id,
        o.order_amount,
        c.customer_segment,
        c.country,
        p.product_category
    FROM silver.orders o
    LEFT JOIN silver.customers c
        ON o.customer_id = c.customer_id
    LEFT JOIN silver.products p
        ON o.product_id = p.product_id
```

Reference joins should happen only when the reference source is stable enough to trust.

5. Conditional business logic

A large amount of real data engineering is business rule implementation.

```
SELECT
    order_id,
    order_amount,
    CASE
        WHEN order_amount >= 10000 THEN 'HIGH_VALUE'
        WHEN order_amount >= 1000 THEN 'MID_VALUE'
        ELSE 'LOW_VALUE'
    END AS order_band
FROM silver.orders
```

The important thing is not the `CASE` statement.

The important thing is where the rule lives, how consistently it is reused, and who owns it.

6. Aggregation

Gold often depends on repeatable aggregations.

```
SELECT
    order_date,
    country,
    COUNT(*)                    AS total_orders,
    SUM(order_amount)           AS total_revenue,
    AVG(order_amount)           AS avg_order_value
FROM silver.orders
GROUP BY order_date, country
```

Aggregation should happen only after the data is trustworthy.

If duplicates exist before this step, your Gold layer lies.

7. Incremental upserts

One of the most important patterns in Level 3 (Transformation Layer).

```
MERGE INTO silver.orders AS target
USING staging_orders AS source
ON target.order_id = source.order_id
WHEN MATCHED THEN UPDATE SET
    target.customer_id = source.customer_id,
    target.order_timestamp = source.order_timestamp,
    target.order_amount = source.order_amount,
    target.status = source.status,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (
    order_id,
    customer_id,
    order_timestamp,
    order_amount,
    status,
    created_at,
    updated_at
)
VALUES (
    source.order_id,
    source.customer_id,
    source.order_timestamp,
    source.order_amount,
    source.status,
    current_timestamp(),
    current_timestamp()
);
```

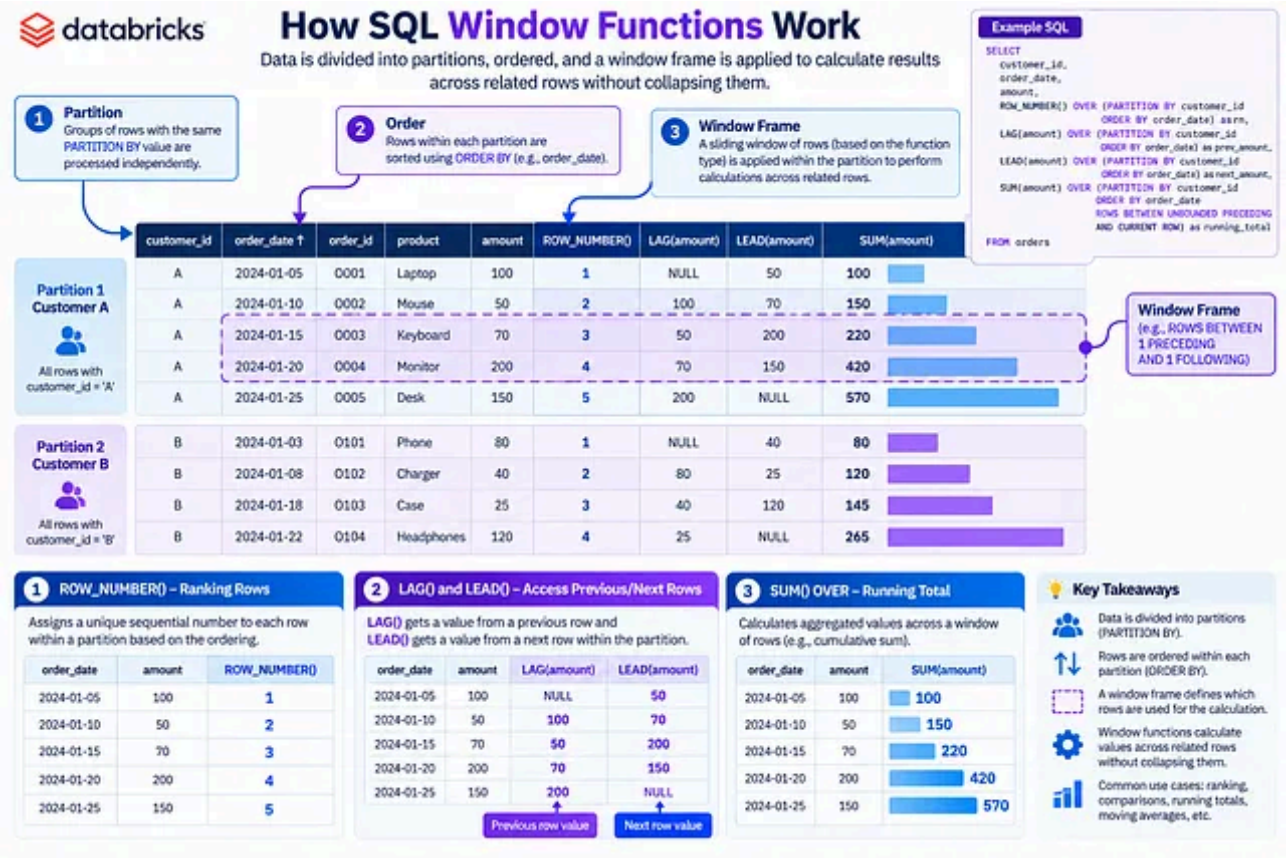
If you're still doing full overwrite for everything, you probably haven't yet matured into strong Level 3 engineering.

Step 4: Window Functions — The Most Important SQL Skill for Data Engineers

If I had to pick one SQL skill that separates average engineers from strong ones in transformation work, it would be window functions.

Why they matter:

- ranking
- deduplication
- sessionization
- lag/lead analysis
- running totals
- change detection
- SCD logic
- time-aware business state analysis



Common window functions you must know

ROW_NUMBER

Used for deduplication and latest-record selection.

```
ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY updated_at DESC
)
```

RANK / DENSE_RANK

Useful when ties matter.

LAG / LEAD

Compare previous and next records.

```
SELECT
    customer_id,
    order_timestamp,
    order_amount,
    LAG(order_amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_timestamp
    ) AS previous_order_amount
FROM silver.orders
```

SUM OVER

Running totals and cumulative behaviour.

```
SELECT
    customer_id,
    order_timestamp,
    order_amount,
    SUM(order_amount) OVER (
        PARTITION BY customer_id
```

```
        ORDER BY order_timestamp
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS lifetime_spend
FROM silver.orders
```

Real-world use case: state change detection

This lets you detect actual transitions instead of repeated status events.

That is the difference between event noise and business meaning.

Step 5: When to Use PySpark Instead of SQL

SQL should be your default for many transformations.

But PySpark becomes the right choice when you need:

1. Complex procedural logic
2. Custom reusable functions
3. Advanced library support
4. Multi-step conditional logic that is awkward in SQL
5. Programmatic generation of transformations
6. More composable engineering patterns
7. Integration with Python-native workflows

Good use cases for PySpark

- schema-driven transformations
- reusable framework code

- feature engineering
- advanced nested data processing
- JSON flattening pipelines
- ML-oriented preparation
- custom scoring logic

Bad use cases for PySpark

- writing 50 lines of DataFrame code for a simple `GROUP BY`
- using Python UDFs where built-in Spark functions would work
- hiding simple business logic inside opaque Python transformations

Example: a clean PySpark transformation

```
from pyspark.sql import functions as F

orders_df = spark.table("bronze.orders")

silver_orders_df = (
    orders_df
    .select(
        F.col("order_id").cast("string").alias("order_id"),
        F.col("customer_id").cast("string").alias("customer_id"),
        F.col("order_timestamp").cast("timestamp").alias("order_timestamp"),
        F.col("order_amount").cast("decimal(12,2)").alias("order_amount"),
        F.upper(F.trim(F.col("status"))).alias("status"),
        F.col("_source_file"),
        F.col("_ingest_timestamp")
    )
    .filter(F.col("order_id").isNotNull())
    .filter(F.col("customer_id").isNotNull())
    .filter(F.col("order_amount") >= 0)
    .filter(F.col("status").isin("PLACED", "SHIPPED", "DELIVERED", "CANCELLED"))
)
```

That is readable.

Example: bad PySpark

- massive nested chained logic
- custom Python UDF for something Spark already provides
- no intermediate naming
- no clear stage boundaries

A good rule:

If SQL expresses it clearly, prefer SQL.
If PySpark makes it clearer, use PySpark.
If Python UDFs are the only way, first ask whether you're making Spark slower than necessary.

Step 6: The Truth About UDFs — Use Carefully

Many engineers reach for UDFs too early.

Why UDFs can be a problem

- harder for Spark optimizer to reason about
- often slower than built-in functions
- harder to debug
- can become black boxes in transformation logic
- Python UDFs introduce serialization overhead

Prefer built-ins whenever possible

Bad instinct:

```
def clean_status(x):
    return x.strip().upper() if x else None
```

Better:

```
F.upper(F.trim(F.col("status")))
```

When UDFs are justified

- complex domain-specific scoring logic
- external library integration
- transformations not supported natively
- business rules that would become unreadable otherwise

If you must use a UDF

- isolate it
- document why built-ins were insufficient
- keep input/output types explicit
- benchmark performance impact
- avoid using it on massive datasets unless justified

Step 7: Incremental Processing — The Heart of Modern Transformation Design

One of the biggest differences between beginner and advanced transformation design is this:

Beginners think in full loads.
Advanced engineers think in incremental state changes.

Why incremental matters:

- lower cost
- faster processing
- lower failure blast radius
- better scalability
- better support for near-real-time systems

Incremental processing patterns

1. append-only event ingestion
2. watermark-based processing
3. latest-record upsert
4. CDC-driven merge
5. SCD Type 1 / Type 2 updates
6. partition-level refresh
7. change-data-feed-driven downstream processing

Basic incremental MERGE pattern

```
MERGE INTO silver.customers AS target
USING staged_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND (
    target.name <> source.name OR
    target.email <> source.email OR
    target.city <> source.city
) THEN UPDATE SET
    target.name = source.name,
    target.email = source.email,
    target.city = source.city,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (
    customer_id,
    name,
    email,
    city,
    created_at,
    updated_at
)
VALUES (
    source.customer_id,
    source.name,
    source.email,
    source.city,
    current_timestamp(),
    current_timestamp()
);
```

Important concept: idempotency

A transformation should produce the same correct final state even if it is run again.

If re-running your pipeline duplicates data, the transformation design is weak.

Idempotency is not optional at Level 3 (Transformation Layer).

Step 8: Handling Late-Arriving and Out-of-Order Data

Real systems are messy.

Events do not always arrive in the order they occurred.

Some arrive:

- late
- duplicated
- corrected
- out of sequence
- backfilled weeks later

A weak transformation layer assumes ideal data.

A mature one assumes reality.

Example problem

An order placed on Monday arrives on Monday.
A cancellation for that order happens on Tuesday but arrives on Friday.
What should the final order state be?

If your logic processes only arrival order, your answer is wrong.



You need to reason using:

- business event timestamp
- ingestion timestamp
- latest valid state logic
- grace periods / watermarks
- correction semantics

Example logic

Keep latest business event per entity:

```
WITH ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY event_timestamp DESC, _ingest_timestamp DESC
        ) AS rn
    FROM bronze.order_events
)
SELECT *
FROM ranked
WHERE rn = 1
```

That does not solve every case, but it is the correct direction.

Questions you must answer

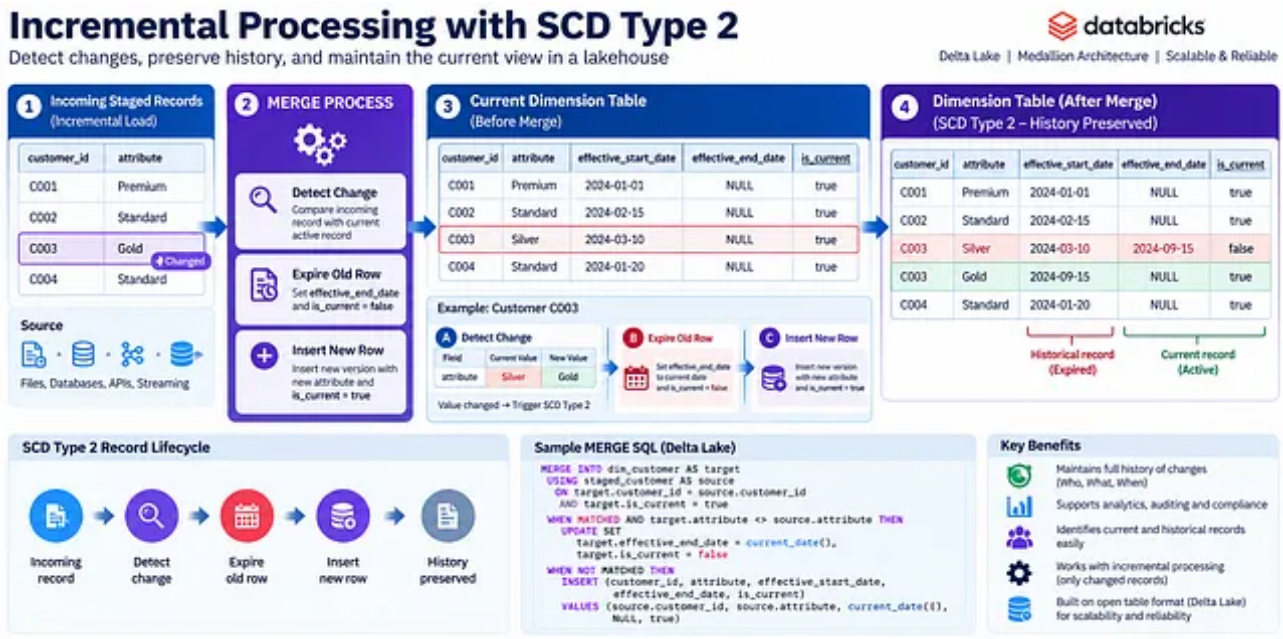
- Which timestamp defines truth: event time or arrival time?
- Can old events overwrite newer records?
- Do we preserve both pre-correction and post-correction states?

- How late is too late?
- Do we re-open prior periods if corrected data arrives?

These are not just technical questions. They are business policy questions implemented technically.

Step 9: Slowly Changing Dimensions (SCD) — Preserving History Correctly

If you work in analytics engineering, finance, customer intelligence, or enterprise reporting, you must understand slowly changing dimensions.



What problem SCD solves

A customer changes city.
A product changes category.
An employee changes department.

Do you:

- overwrite the old value?
- keep both old and new values?
- track effective dates?
- preserve historic reporting consistency?

That is what SCD addresses.

SCD Type 1

Overwrite old value. No history preserved.

Use when:

- correction is more important than history
- the old value is not analytically meaningful
- dimension history is not required

```
MERGE INTO silver.customers AS target
USING staged_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN UPDATE SET
    target.city = source.city,
    target.email = source.email,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT *
```

Simple, efficient, but no history.

SCD Type 2

Preserve history by creating new versions of a record.

Typical columns:

- `effective_start_date`
- `effective_end_date`
- `is_current`

Example conceptual structure

customer_id	city	effective_start_date	effective_end_date	is_current
C101	Pune	2024-01-01	2025-02-15	false
C101	Berlin	2025-02-16	null	true

This allows you to answer:

- Where did the customer live when the order happened?
- What was the customer segment at the time of the transaction?

Typical SCD Type 2 flow

1. detect changed records
2. expire current active row
3. insert a new active row

Pseudo-SQL idea

```
-- Step 1: expire changed current rows
UPDATE silver.customer_dim
SET effective_end_date = current_date() - 1,
    is_current = false
WHERE customer_id IN (
    SELECT s.customer_id
    FROM staged_customers s
    JOIN silver.customer_dim d
      ON s.customer_id = d.customer_id
    WHERE d.is_current = true
      AND (
          s.city <> d.city OR
          s.email <> d.email
      )
);

-- Step 2: insert new current rows
INSERT INTO silver.customer_dim (
    customer_id,
    city,
    email,
    effective_start_date,
    effective_end_date,
    is_current
)
SELECT
    customer_id,
    city,
    email,
    current_date(),
    NULL,
    true
FROM staged_customers;
```

In production, you tighten this logic to avoid unnecessary inserts.

Interview reality

If asked “Explain SCD Type 1 vs Type 2,” do not stop at definitions.

Explain:

- when to use each
- trade-offs
- storage impact
- reporting impact
- how historical joins work
- why Type 2 matters for facts tied to time

That shows depth.

Step 10: Dimensional Modelling — Facts, Dimensions, and Business Readability

A lakehouse does not eliminate the need for modelling.

In fact, modelling becomes more important because flexibility can easily become ambiguity.

Fact tables

Contain measurable business events:

- orders
- payments
- clicks
- shipments

- transactions

Typical columns:

- business keys
- foreign keys to dimensions
- metrics
- timestamps

Dimension tables

Contain descriptive business context:

- customer
- product
- region
- date
- employee

Typical columns:

- surrogate/business key
- attributes
- hierarchy columns
- SCD metadata if historical

Star schema

A central fact table with surrounding dimensions.

Why it remains powerful:

- intuitive for analysts
- efficient for BI
- supports reusable metrics
- easier business interpretation

Example

`fact_orders`

- order_id
- customer_id
- product_id
- order_date_key
- order_amount
- quantity

`dim_customer`

- customer_id
- customer_name
- customer_segment
- city
- country

```
dim_product
```

- product_id
- product_name
- brand
- category

Why Gold still matters

Some people say, “We have Delta Lake and compute on demand; why model at all?”

Because raw flexibility does not equal business usability.

A dashboard team should not reverse-engineer event semantics from raw event streams.

That is your job in Level 3 (Transformation Layer).

Step 11: Joining Data Correctly — One of the Most Dangerous Areas in Transformation Design

A huge percentage of wrong business numbers come from bad joins.

Common join mistakes

1. joining on non-unique keys
2. joining before deduplication
3. mismatched granularity

4. inner join where left join was needed
5. accidental many-to-many explosion
6. joining facts to facts without clear grain strategy

Example of a dangerous join

If `customers` has duplicate `customer_id` records and you join it directly to orders, revenue may double or triple.

Good discipline before joins

Ask these every time:

- What is the grain of the left table?
- What is the grain of the right table?
- Is the join key unique on the dimension side?
- Can this multiply rows?
- Should I deduplicate before joining?
- Should I aggregate before joining?

Protective SQL habit

```sql
SELECT customer_id, COUNT(*)
FROM silver.customers
GROUP BY customer_id
HAVING COUNT(*) > 1
```

Run this before trusting the join.

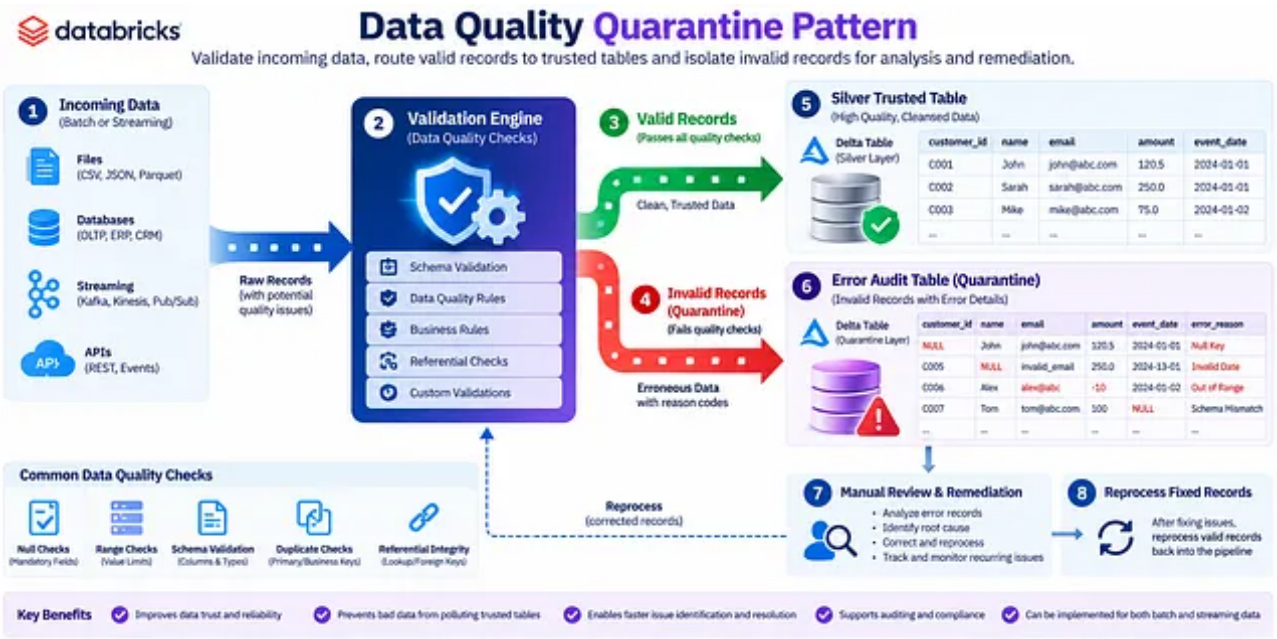
This one habit prevents a surprising number of reporting disasters.

Step 12: Data Quality Inside Transformation Logic

A lot of teams treat data quality as a separate concern.

That is a mistake.

At Level 3 (Transformation Layer), data quality is embedded into transformation logic itself.



Examples of transformation-layer quality checks

- mandatory key presence
- valid status values
- positive transaction amounts
- reference integrity

- uniqueness assumptions
- date sanity checks
- duplicate thresholds
- null thresholds

Example: validation output pattern

Instead of silently dropping bad records, consider splitting them:

- valid records → Silver trusted table
- invalid records → quarantine/error table

```
CREATE OR REPLACE TEMP VIEW validated_orders AS
SELECT *,
       CASE
           WHEN order_id IS NULL THEN 'MISSING_ORDER_ID'
           WHEN customer_id IS NULL THEN 'MISSING_CUSTOMER_ID'
           WHEN order_amount < 0 THEN 'NEGATIVE_AMOUNT'
           ELSE NULL
       END AS validation_error
FROM bronze.orders;
```

Then:

```
INSERT INTO silver.orders_valid
SELECT * FROM validated_orders
WHERE validation_error IS NULL;

INSERT INTO silver.orders_invalid
SELECT * FROM validated_orders
WHERE validation_error IS NOT NULL;
```

This is much better than invisibly dropping records and pretending they never existed.

Step 13: Designing Transformations for Reusability, Not Notebook Chaos

Many Databricks projects become hard to maintain because transformation logic lives in scattered notebooks with copied code.

That is not maturity.

Strong Level 3 (Transformation Layer) design means:

- common naming conventions
- reusable transformation modules
- clear separation of SQL and orchestration
- parameter-driven execution
- version-controlled logic
- testable units
- consistent layer contracts

What good transformation organisation looks like

- one notebook/script per logical transformation unit
- standardised input/output contracts
- no hidden side effects
- no hardcoded environment-specific paths

- transformation names based on business meaning

Bad:

- `notebook_final_v7_use_this_revised`
- same deduplication logic repeated in 5 places
- different definitions of active customer across teams

Good:

- `silver_customer_current`
- `silver_orders_conformed`
- `gold_daily_revenue_by_region`
- `gold_customer_lifetime_value`

Names should communicate meaning, not author memory.

Step 14: Performance Thinking Inside Transformations

Level 6 (Performance & Cost Optimisation) will go much deeper into tuning, but good transformation engineers already think about performance at Level 3 (Transformation Layer).

High-impact habits now

1. Filter early when possible
2. Project only needed columns
3. Avoid unnecessary wide shuffles

4. Deduplicate before expensive joins when feasible
5. Avoid repeated scans of the same large source
6. Prefer built-in functions over UDFs
7. Use incremental processing instead of full reloads
8. Understand join strategy implications

Example

Bad:

- read full table
- join everything
- filter later
- aggregate at the end

Better:

- filter source early
- type/validate early
- deduplicate before join
- join only necessary dimensions
- aggregate once trust is established

Transformation quality is not just correctness.

It is correctness at sustainable cost.

Step 15: A Complete Example — Bronze to Silver to Gold

Let's tie everything together.

Bronze: raw orders landed from Level 2 (Lakehouse Foundations)

Columns:

- raw order attributes
- `_source_file`
- `_ingest_timestamp`
- `_batch_id`

Silver objective

Produce one trusted order record per `order_id`.

Requirements:

- cast types
- clean status
- remove invalid rows
- deduplicate by latest ingest timestamp
- enrich with customer segment

Silver SQL example

```
WITH typed AS (
    SELECT
        CAST(order_id AS STRING) AS order_id,
        CAST(customer_id AS STRING) AS customer_id,
        CAST(product_id AS STRING) AS product_id,
        CAST(order_timestamp AS TIMESTAMP) AS order_timestamp,
        CAST(order_amount AS DECIMAL(12,2)) AS order_amount,
        UPPER(TRIM(status)) AS status,
        _source_file,
        _ingest_timestamp,
        _batch_id
    FROM bronze.orders
),

validated AS (
    SELECT *
    FROM typed
    WHERE order_id IS NOT NULL
      AND customer_id IS NOT NULL
      AND order_timestamp IS NOT NULL
      AND order_amount >= 0
      AND status IN ('PLACED', 'SHIPPED', 'DELIVERED', 'CANCELLED')
),

deduplicated AS (
    SELECT *
    FROM (
        SELECT
            *,
            ROW_NUMBER() OVER (
                PARTITION BY order_id
                ORDER BY _ingest_timestamp DESC
            ) AS rn
        FROM validated
    ) x
    WHERE rn = 1
),

enriched AS (
    SELECT
        d.order_id,
        d.customer_id,
        d.product_id,
        d.order_timestamp,
        d.order_amount,
        d.status,
        c.customer_segment,
        c.country,
        d._source_file,
        d._ingest_timestamp,
        d._batch_id
```

```
    FROM deduplicated d
    LEFT JOIN silver.customers_current c
        ON d.customer_id = c.customer_id
)

SELECT *
FROM enriched
```

Gold objective

Daily revenue by country and segment.

```
SELECT
    DATE(order_timestamp) AS order_date,
    country,
    customer_segment,
    COUNT(*) AS total_orders,
    SUM(order_amount) AS total_revenue,
    AVG(order_amount) AS avg_order_value
FROM silver.orders_conformed
GROUP BY
    DATE(order_timestamp),
    country,
    customer_segment
```

That is a good example of proper layer behaviour:

- Bronze stores reality
- Silver creates trust
- Gold creates usability

Step 16: Interview Questions You Will Actually Be Asked at Level 3 (Transformation Layer)

Here are the questions that come up again and again:

Question	What a strong answer should include
What belongs in Bronze vs Silver vs Gold?	Bronze raw + provenance, Silver trusted/conformed entity layer, Gold consumption-ready marts
SQL or PySpark — how do you choose?	Prefer SQL for clarity and standard transformations; PySpark for procedural complexity, framework design, nested data, advanced logic
How do you deduplicate records?	Window functions, survivorship rules, business key clarity, ordering logic
What is idempotent transformation design?	Re-runs should not create incorrect duplicates or drift in final state
Explain `MERGE` and why it matters.	Atomic upsert, incremental processing, update/insert handling
What is late-arriving data?	Data whose event time is older than its arrival time; explain business vs ingest timestamp
Explain SCD Type 1 vs Type 2.	Overwrite vs historical preservation, effective dates, analytics implications
How do you prevent bad joins?	Grain awareness, uniqueness checks, deduplication, aggregation before join if needed
When would you avoid UDFs?	Performance, optimizer visibility, serialization overhead
How do you make transformation logic trustworthy?	Validation, quarantine strategy, tested logic, clear ownership, stable semantics

If you can answer these with examples, not definitions alone, you're moving into senior territory.

Step 17: Common Mistakes in Level 3 (Transformation Layer)

Mistake	Why it hurts
Putting business logic in Bronze	Destroys replayability and confuses layer ownership
Using Gold as a cleanup layer	Means Silver is not actually trustworthy
Full-refreshing everything	Expensive, slow, and unnecessary at scale
Blind deduplication	Can delete the wrong record without survivorship logic
Overusing Python UDFs	Slows jobs and hides logic
Joining before checking grain	Causes row explosion and false metrics
Replacing nulls carelessly	Creates fake certainty
Not preserving history where needed	Breaks time-based reporting
Mixing business definitions across teams	Creates inconsistent metrics
No invalid-record strategy	Bad data either disappears or pollutes trusted tables
SQL notebooks with no structure	Hard to debug, review, and maintain
Treating transformation as "just coding"	Ignores business semantics, which is the real job

Step 18: What Good Looks Like at This Level

You know you are strong at Level 3 (Transformation Layer) when:

- You can explain what belongs in Bronze, Silver, and Gold without confusion
- You write transformations with clear stages and naming
- You use SQL confidently and PySpark intentionally
- You understand deduplication as survivorship, not just row removal
- You build idempotent incremental pipelines
- You handle late-arriving data thoughtfully

- You know when to preserve history and when not to
- You model trusted business-ready datasets
- You think about correctness, readability, and cost together
- Other teams trust the datasets your transformations produce

That last point matters most.

A transformation layer is successful when people stop debating the numbers and start using them.

Conclusion

Level 3 (Transformation Layer) is where data engineering becomes meaning engineering.

At Level 2 (Lakehouse Foundations), you built trustworthy storage, medallion structure, and ingestion patterns. But storage alone does not create value. Transformation creates value.

This is where you take raw records and turn them into:

- trusted entities
- stable business definitions
- reusable metrics
- historically correct dimensions
- incremental models that scale
- datasets other teams can actually depend on

And this is also where a data engineer's maturity becomes visible.

Not in how many APIs they know.
Not in how many notebooks they can run.
But in whether they can design transformation logic that is:

- correct
- understandable
- repeatable
- scalable
- and aligned with business meaning

That is the real work of Level 3 (Transformation Layer).

If Level 2 protected your platform from becoming a swamp, Level 3 protects your company from making decisions on broken truth.

Wait for the Next Detailed Blog

Next Monday, 11 AM IST
We will go deep into Level 4 (Pipeline Engineering):

- Databricks Workflows
- task orchestration
- retries and failure handling
- dependency design
- Delta Live Tables / Lakeflow Declarative Pipelines
- monitoring and alerting